

Rapport de tests — capacité, mémoire, vulnérabilités

Tests exécutés sur les binaires réels (Linux, HTTPS), reproductibles avec `go test ./...`.

Auteur : Dioubate Cheick · cheickdioubate2@gmail.com · Méthode : banc d'essai automatisé (central + agents réels) + suite de tests unitaires et d'intégration.

Synthèse

Machine de test : 2 vCPU, Linux, disque local. Central et agents sur la même machine (ils se partagent donc les 2 cœurs). Sur un serveur plus puissant, les débits de traitement augmentent avec le nombre de cœurs.

~22 000

événements/s traités
et stockés (2 vCPU)

1 000 000

événements
envoyés, 1 000 000
stockés, 0 perte

1 000

agents simultanés, 0
perte

~440 Mo

RAM du central
après 1 M
d'événements

Deux chiffres à ne pas confondre. L'ancienne version de ce rapport annonçait 131 000 événements/s : c'était la vitesse à laquelle le central *accepte* des lots (réponse HTTP), mesurée en mémoire, pas la vitesse à laquelle il les *traite* (géolocalisation, renseignement par IP, corrélation, écriture disque). Ce chiffre était trompeur ; il est retiré. Tous les chiffres ci-dessous sont mesurés de bout en bout : un événement n'est compté que lorsqu'il est écrit sur le disque du central.

1. Débit de bout en bout (central réel, HTTPS)

Chaque agent simulé s'inscrit réellement (jeton + empreinte machine), puis envoie 5 lots de 100 événements en parallèle. Un lot refusé pour saturation (HTTP 503) est renvoyé par l'agent : le central n'accuse réception qu'une fois le lot entièrement pris en charge.

AGENTS SIMULTANÉS	ÉVÉNEMENTS	ACCEPTÉS/S	TRAITÉS ET STOCKÉS/S	LATENCE D'ENVOI P50 / P95	RAM CENTRAL	PERTE
40	20 000	30 648	17 723	200 / 478 ms	135 Mo	0
100	50 000	22 118	18 294	601 / 776 ms	324 Mo	0
500	250 000	21 560	20 713	587 / 728 ms	368 Mo	0
1 000	500 000	21 518	21 030	591 / 725 ms	390 Mo	0

Un million d'événements (100 agents, lots de 1 000)

MESURE	RÉSULTAT
Acceptés	1 000 000 en 41,4 s (24 155/s)
Traités et stockés	1 000 000 / 1 000 000 en 44,6 s (22 458/s) – 0 perte, 0 lot rejeté définitivement
Disque (JSONL)	445 Mo pour 1 M d'événements (~450 octets/événement, avant compression)
RAM du central	435 Mo (pic 461 Mo)
Recherche ciblée <code>ip:41.*</code> sur 1 M	5 776 résultats en 1,3 s
Recherche large <code>sev:yellow</code> (1 M résultats)	8,9 s (comptage + facettes + histogramme sur tout le volume)
Tableau de bord	0,4 à 0,6 s

Le moteur interne seul (sans réseau) traite **~43 000 événements/s** sur ces 2 vCPU. La recherche lit les fichiers JSONL en parallèle avec un décodeur dédié et un préfiltre : les recherches ciblées (IP, sévérité, catégorie) restent rapides ; une recherche qui correspond à des millions d'événements parcourt tout le volume, il n'y a pas d'index inversé.

Un seul agent réel (fichier journal → central, bout en bout)

Un vrai agent lit un vrai fichier journal où l'on écrit d'un coup 20 000 puis 80 000 lignes (250 IP attaquantes). Chaque ligne est comptée côté central, soit comme événement brut, soit dans un agrégat.

MODE DE LIMITATION	RAFALE	RÉSULTAT AU CENTRAL	COMPTABILISÉS
aggregate (défaut)	20 000	15 000 bruts + 250 agrégats (5 000 occurrences)	20 000 / 20 000
aggregate	80 000	250 agrégats (80 000 occurrences, avec première/dernière occurrence)	80 000 / 80 000
queue	20 000	20 000 bruts en 5 s (~4 200/s)	20 000 / 20 000
queue	80 000	80 000 bruts en 11 s (~7 500/s)	80 000 / 80 000

Plus aucun événement « `rate_limited` » perdu. Au-delà de 60 occurrences par minute d'une même règle pour une même IP, l'agent n'envoie plus chaque ligne mais un événement de synthèse `aggregated=oui, count, first_seen, last_seen` (mode `aggregate`); les statistiques du central pondèrent ce compteur. En mode `queue`, tout est envoyé en brut, le surplus passant par le tampon disque de l'agent (200 Mo max) et n'étant supprimé qu'après accusé de réception. Les alertes rouges passent par une file prioritaire séparée.

Bannissement massif

IP BANNIES	IMPORT GROUPE	SYNCHRO DE 100 AGENTS P50 / P95	LISTE COMPRESSÉE
1 000	15 ms	28 / 44 ms	2 Ko
10 000	11 ms	58 / 88 ms	22 Ko
100 000	305 ms	546 ms / 1,7 s	217 Ko

L'ajout unitaire depuis la console (une entrée d'audit scellée par IP) plafonne à ~280/s : pour de gros volumes, utiliser l'import groupé `/api/bans/bulk`.

Battements de cœur (5 000 agents)

Test séparé, sans événements : 5 000 agents enrôlés en moins de 4 minutes ; avec un battement toutes les 10 s (500 req/s), le central en soutient 975/s même quand chaque agent recharge toute la liste de blocage. Ce test ne mesure **pas** l'ingestion d'événements de 5 000 agents : la capacité d'événements vérifiée est celle du tableau ci-dessus (1 000 agents, ~21 000 évt/s).

2. Consommation de l'agent & zéro perte

MESURE	VALEUR
Agent au repos	22 Mo
Agent après 100 000 lignes analysées	42 Mo (pic 55 Mo)
Coupure du central → mise en tampon disque	oui (aucune perte)
Retour du central → renvoi du tampon	5 000 / 5 000 événements
Accusé de réception partiel	seuls les événements acquittés sont retirés du tampon

3. Tests de vulnérabilité

39 contrôles exécutés contre le binaire réel en HTTPS. **Tous réussis.**

- ✓ TLS 1.0 / 1.1 refusés, 1.2 / 1.3 acceptés
- ✓ En-têtes HSTS, CSP, X-Frame-Options, nosniff, Referrer-Policy
- ✓ Toutes les API refusent l'accès sans session (401)
- ✓ Mots de passe triviaux (admin / password / 123456) refusés
- ✓ Cookie de session HttpOnly + Secure + SameSite=Strict
- ✓ CSRF : écriture sans en-tête ou cross-origin refusée
- ✓ Usurpation d'agent (id B + clé A) refusée
- ✓ Clé d'agent rejouée depuis une autre machine refusée
- ✓ Battement de cœur / événements sans clé refusés
- ✓ JSON malformé rejeté proprement (400)
- ✓ Corps de 30 Mo rejeté sans saturer la mémoire (+2 Mo)
- ✓ En-têtes de 100 Ko refusés (431)
- ✓ Traversée de chemin (8 variantes) bloquée
- ✓ Fichiers sensibles (clé, comptes) inaccessibles
- ✓ Méthode TRACE refusée
- ✓ XSS stocké servi en JSON, jamais en HTML (CSP sans script inline)
- ✓ Manifeste de mise à jour falsifié refusé (signature)
- ✓ Slowloris : connexion lente coupée par le central
- ✓ Force brute du jeton d'enrôlement : IP bloquée (429)
- ✓ Force brute console : IP verrouillée (429)

Mémoire maximale du central pendant l'ensemble des attaques : **139 Mo**. Aucun plantage, aucune fuite mémoire.

4. Corrections apportées après l'audit

CONSTAT	CORRECTION	VÉRIFICATION
Événements au-delà de la limite de débit jetés (20 000 → 5 000 perdus ; 80 000 → 65 000 perdus)	Agrégation comptée ou file disque, ACK avant suppression, file prioritaire rouge	20 000/20 000 et 80 000/80 000
Chiffre de 131 000 évt/s trompeur (acceptation seule)	Retiré ; mesures de bout en bout uniquement	~22 000/s stockés
Traitement central lent (~6 500 évt/s en interne)	Fenêtres glissantes incrémentales, index géographique clairsemé, pipeline par lots, écritures différées	~43 000/s en interne
Recherche lente sur gros volume (9 s pour une IP sur 1 M)	Lecture parallèle, décodeur JSON dédié, préfiltre	1,3 s
Mémoire non bornée avec des millions d'IP	Plafonds (50 000 IP suivies, éviction), limite mémoire souple 512 Mo	RAM < 470 Mo à 1 M
Ancien script d'installation dans le paquet	Script unique, identique dans le paquet et le dépôt	contrôlé

5. Limites connues (à lire avant d'évaluer)

- Un seul central, pas de grappe ni de réplication.
- Stockage JSONL par jour, non compressé (~450 o/événement) ; rétention en jours à régler.
- Recherche par balayage, sans index inversé : rapide si ciblée, plusieurs secondes si elle couvre des millions d'événements.
- Pas de collecte syslog réseau ni d'agent macOS.
- Débits mesurés sur 2 vCPU partagés entre central et agents simulés.
- Le téléchargement de l'agent est public par conception ; seul l'enrôlement (jeton + approbation) donne accès.

6. Reconnaissance des agents (anti-clonage)

Chaque agent est lié, dès l'enrôlement, à une **empreinte matérielle** de sa machine (identifiant système + adresses matérielles). À chaque échange, le central vérifie cette empreinte : une clé d'agent volée et rejouée depuis une **autre machine** produit une empreinte différente, la requête est **refusée** et une alerte rouge `SYS-AGENT-CLONE` est levée. Un agent fictif sans jeton d'enrôlement valide ne peut de toute façon pas s'enregistrer.